

Aletheia — Benchmark Final

993 bases, 54 transformações, 53.622 amostras — relatório completo de execução e resultados

waizbart · relatório gerado por Claude Code

3 de outubro de 2026

- [Sumário executivo](#)
- [1. Objetivo e desenho do experimento](#)
 - [1.1 A pergunta](#)
 - [1.2 Os três níveis de medição](#)
 - [1.3 A taxonomia de transformações](#)
- [2. Ambiente e congelamento](#)
 - [2.1 Congelamento do código](#)
 - [2.2 Ambiente de execução](#)
- [3. Desvios em relação ao roteiro](#)
 - [3.1 Execução em contêiner, não no host](#)
 - [3.2 Quatro núcleos, não oito](#)
 - [3.3 Manifest regerado, variantes reaproveitadas](#)
 - [3.4 Passo 4 sem `docker compose down -v`](#)
- [4. Passo 1 — o dataset](#)
 - [4.1 Geração](#)
 - [4.2 Metadados do manifest](#)
 - [4.3 Tempos de execução observados](#)
 - [4.4 Bases sem features ORB](#)
- [5. Mudanças de código antes do congelamento](#)
 - [5.1 Correção: decodificadores de BMP, TIFF e WebP no pHash](#)
 - [5.2 Caracterização: o resíduo de cor das rotações cardeais](#)
- [6. Instrumentação: como o experimento mede](#)
 - [6.1 Fidelidade ao caminho de produção](#)
 - [6.2 O repositório em memória](#)
 - [6.3 Diferenças conhecidas em relação à API real](#)
 - [6.4 O controle negativo acrescentado](#)
 - [6.5 Colunas registradas por amostra](#)
 - [6.6 Comandos executados](#)
- [7. Passo 3 — resultados de acurácia](#)
 - [7.1 Pipeline completo](#)
 - [7.2 Par a par, para leitura lado a lado](#)

- 7.3 Robustez do resultado a duas objeções
- 7.4 Análise dos falsos positivos
- 7.5 Análise dos falsos negativos
- 7.6 O pré-filtro — a origem da perda de recall
- 7.7 Estágio que decidiu
- 7.8 Controles negativos
- 7.9 Tabela completa por transformação
- 8. Passo 3 — resultados de latência
 - 8.1 O achado: o pHash domina a latência
 - 8.2 Consultas que casam versus que não casam
- 9. Passo 2 — verificação cruzada par a par
 - 9.1 Resultado
 - 9.2 A verificação cruzada: concordância exata
 - 9.3 Por que o teste termina em FAIL
 - 9.4 O que as três concordâncias estabelecem, em conjunto
- 10. Achado do dataset: duas bases são a mesma fotografia
 - 10.1 Como apareceu
 - 10.2 A evidência
 - 10.3 Consequências para o relato
 - 10.4 A limitação metodológica a declarar
- 11. Defeitos encontrados nas ferramentas de medição
 - 11.1 `TestDataset_FullMatrix` abortava na primeira base sem features
 - 11.2 O resumo conflava dois eventos opostos
- 12. Passo 4 — end-to-end contra a API
 - 12.1 Concordância com a simulação: exata
 - 12.2 O que essa concordância estabelece
 - 12.3 Contrapartida: custo da pilha completa
- 13. Achados e recomendações consolidados
 - 13.1 O que o sistema faz bem
 - 13.2 Recomendações, em ordem de retorno
 - 13.3 O que não deve ser feito
- 14. Limitações e ameaças à validade
- 15. Procedência dos arquivos
 - 15.1 Reprodução

Sumário executivo

O benchmark final do sistema Aletheia foi executado sobre **993 imagens base** do Lorem Picsum, às quais foram aplicadas as **54 transformações** da taxonomia, produzindo **53.622**

amostras rotuladas, mais **993 controles negativos sintéticos** acrescentados por este experimento. A medição ocorreu em três níveis: comparação par a par isolada, pipeline completo de verificação, e end-to-end através da API HTTP com PostgreSQL real.

Resultado principal. No estrato de alta confiança, o pipeline completo atingiu precisão **1,000** (IC 95%: 0,999–1,000) e recall **0,983** (IC 95%: 0,980–0,985), com **3 falsos positivos em 8.786 predições positivas**. Para um certificador, a precisão é a métrica que importa: um falso positivo autentica uma falsificação.

Métricas por estrato, pipeline completo:

estrato	n	precisão	recall
Alta confiança	16.881	1,000 [0,999, 1,000]	0,983 [0,980, 0,985]
Limítrofe	36.741	0,893 [0,889, 0,898]	0,611 [0,605, 0,617]
Todas	53.622	0,929 [0,925, 0,932]	0,706 [0,702, 0,711]

As três medições concordam exatamente. O protocolo admitia divergência de até 1 ponto percentual entre os níveis; a divergência observada foi **zero**. O pipeline medido em memória (passo 3) e através da API HTTP com PostgreSQL real (passo 4) produziram matrizes de confusão idênticas nos três estratos **e nas 21 famílias, uma por uma**. A medição par a par da ferramenta original (passo 2) reproduziu a coluna correspondente da simulação, igualmente dígito por dígito.

Oito achados que o relatório detalha:

1. **O pré-filtro perceptual é o mecanismo dominante de falso negativo.** Em 4.206 amostras positivas o certificado correto nunca chegou à correspondência visual, e **3.721 delas teriam casado par a par**. São falsos negativos inteiramente atribuíveis à busca, não ao matcher.
2. **A janela top-64 nunca perdeu nada.** A mediana é de 1 candidato por consulta, p95 de 3, máximo de 12. O `verifyTopK` está ordens de magnitude acima do necessário, e o limite real é o corte de Hamming.
3. **O pHash consome 75% da latência de verificação** (218 ms de 289 ms de mediana), por uma DCT 2D de implementação ingênua.
4. **A assimetria das rotações cardeais não é erro de warp.** É limite de precisão do alinhamento geométrico, demonstrado substituindo a homografia estimada pela analítica exata.
5. **Os três falsos positivos de alta confiança são todos `sepia`**, com resíduo logo abaixo dos dois limiares de cor.
6. **Duas bases do Picsum são a mesma fotografia** sob IDs diferentes, o que explica integralmente as 27 atribuições a "outro certificado" e os 2 acertos no controle negativo.
7. **A definição do estrato limítrofe não enviesou o resultado.** Excluindo as 100 bases que a informaram, a manchete é idêntica até a terceira decimal.
8. **No estrato limítrofe há uma inversão de monotonicidade reveladora:**
`saturation_boost_1.5x` erra mais que `2.0x` (0,655 contra 0,188) e `hue_shift_30deg` mais que `180deg` (0,240 contra 0,085). Quanto menor a edição, mais o sistema a considera o

mesmo conteúdo — comportamento desejado num verificador de identidade, que põe em dúvida o rótulo, não o sistema.

Uma correção de bug entrou antes do congelamento: os decodificadores de BMP, TIFF e WebP no cálculo do pHash. **Nenhum limiar de decisão foi alterado em nenhum momento.**

1. Objetivo e desenho do experimento

1.1 A pergunta

O Aletheia certifica imagens e depois verifica se uma imagem apresentada corresponde a algum certificado emitido. A pergunta do benchmark é quão bem ele faz isso quando o banco contém mil certificados, e onde exatamente o sistema falha quando falha.

A métrica principal é a **precisão no estrato de alta confiança**. O raciocínio é o uso: num certificador, um falso positivo autentica uma falsificação, o que é um erro qualitativamente pior que um falso negativo, que apenas deixa de reconhecer um original.

1.2 Os três níveis de medição

Nível	O que isola	Teste
Par a par	ORB, RANSAC, resíduo de cor e cobertura contra a base correta , entregue de mão beijada	<code>TestDataset_FullMatrix</code> e a coluna <code>pair_matched</code> do experimento
Pipeline completo	O caminho inteiro: SHA-256, pré-filtro LSH, correspondência visual contra um banco com as 993 bases	<code>TestTCC_PipelineSimulation</code>
End-to-end	O mesmo pipeline através da API HTTP, com PostgreSQL real	<code>TestE2E_GeneratedDataset_Matrix</code>

A separação entre os dois primeiros níveis é o ponto central do desenho. Uma comparação par a par recebe a referência correta; o pipeline tem de **encontrá-la** entre 993 certificados. A diferença entre as duas colunas é o custo do pré-filtro, e é exatamente onde o sistema perde mais.

1.3 A taxonomia de transformações

As 54 transformações dividem-se em dois grupos semânticos, rotulados por **intenção**, não pelo que o matcher faz:

- **Preservam identidade** (`expected_match: true`), 36 transformações: recompressão JPEG (6 níveis), redução de escala (5), ampliação (2), troca de formato (4), rotação cardeal (3),

rotação de ângulo pequeno (3), corte de borda (3), brilho (4), ruído gaussiano (2), nitidez (1), recompressão estilo WhatsApp (1), P3 tratado como sRGB (1).

- **Quebram identidade** (`expected_match: false`), 18 transformações: escala de cinza, sépia, deslocamento de matiz (4), aumento de saturação (2), inversão de cor, recolorização localizada, sobreposição de conteúdo (4), corte agressivo (3), corte de borda a 20%, e imagem diferente.

Cada transformação carrega um **estrato de confiança**: `high` quando o rótulo está longe da fronteira de decisão, `borderline` quando está perto. Os gates duros usam apenas o estrato de alta confiança.

A taxonomia tem mais positivos que negativos — 34.755 contra 18.867 — então a acurácia está inflada por construção e não deve ser reportada isoladamente.

2. Ambiente e congelamento

2.1 Congelamento do código

O benchmark só vale como teste se código, limiares e rótulos forem fixados antes de qualquer resultado.

item	valor
Tag	<code>tcc-benchmark-v1</code>
Commit	<code>361c65d8001c996aa76f70c732bcf6afa1797718</code>
Branch	<code>tcc/benchmark-v1</code>
Pull request	<code>waizbart/aletheia-api#23</code>

Limiares congelados, conferidos pelo próprio teste contra o `thresholds_snapshot` do manifest — o experimento se recusa a rodar se divergirem:

limiar	constante	valor
Mínimo de inliers do RANSAC	<code>MinInliers</code>	20
Resíduo LAB médio máximo	<code>MaxColorMean</code>	8,0
Resíduo LAB máximo por célula	<code>MaxCellDist</code>	38,0
Cobertura mínima da área	<code>MinAreaCoverage</code>	0,42
Distância pHash máxima	<code>MaxPHashDistance</code>	96
Janela de candidatos	<code>verifyTopK</code>	64

Nenhum destes valores foi alterado em nenhum momento, antes ou depois de ver resultados.

Commits posteriores à tag

Um commit entrou depois do congelamento, `5a36615`. Ele **não toca o sistema sob teste**: corrige o `TestDataset_FullMatrix`, que é a verificação cruzada opcional do passo 2, e o `scripts/tcc_summary.py`, que apenas formata um CSV. O código do pipeline medido continua sendo o da tag. A seção 11 detalha os dois defeitos.

2.2 Ambiente de execução

item	valor
Go	1.25.14 linux/amd64
OpenCV	4.6.0 (via <code>pkg-config</code>)
gcc	12.2.0 (Debian 12.2.0-14+deb12u1)
CGO	habilitado
CPU	AMD EPYC-Rome, 4 vCPU , 1 thread por núcleo
Cache	L1d/L1i 128 KiB, L2 2 MiB, L3 16 MiB
RAM	7,6 GiB
Kernel	Linux 6.8.0-106-generic, Ubuntu 24.04.4 LTS
Virtualização	KVM
Disco livre	34 GiB
Docker	29.4.2; Compose v5.1.3
Python	3.12.3
PostgreSQL	16-alpine (contêiner)

O arquivo `results/ambiente.txt` traz a saída bruta de cada comando de levantamento.

3. Desvios em relação ao roteiro

Quatro desvios, todos por restrição do ambiente ou por economia de execução. Nenhum altera o sistema sob teste.

3.1 Execução em contêiner, não no host

O host **não tem Go, gcc, `pkg-config` nem OpenCV**, e o `sudo` exige senha, de modo que instalar a toolchain não era possível. Toda a execução ocorreu num contêiner construído a partir do próprio `Dockerfile` do repositório, que usa `golang:1.25-bookworm` mais `libopencv-dev` — fornecendo exatamente Go 1.25 e OpenCV 4.6, as versões que o roteiro exige, e as mesmas contra as quais o `gocv` v0.31.0 compila.

O repositório foi montado **no mesmo caminho absoluto do host**, `/home/waizbart/projects/aletheia-api`, porque o manifest guarda caminhos absolutos e o `TestDataset_FullMatrix` os lê sem remapear. Montar em outro prefixo quebraria o passo 2.

3.2 Quatro núcleos, não oito

O roteiro extrapolou tempos para 8 núcleos; a máquina tem 4. Os tempos praticamente dobram, e os valores observados estão na seção 4.3. Por ser um servidor e não um notebook, não houve risco de suspensão, e o `systemd-inhibit` foi dispensado.

Os três níveis foram encadeados **em sequência, nunca em paralelo**, por duas razões: a rodada de latência precisa de uma thread sem disputa de CPU, e o cliente HTTP do teste `e2e` tem timeout de 60 segundos, que a contenção poderia estourar — e um único erro HTTP derruba o relatório do `e2e`.

3.3 Manifest regerado, variantes reaproveitadas

O dataset de 993 bases já existia em disco, de uma execução anterior, mas o `manifest.json` estava defasado em três pontos:

1. Caminhos absolutos de outro prefixo de montagem (`/work/...`), inservíveis para o passo 2.
2. `thresholds_snapshot` sem `min_area_coverage` — gerado antes de a constante existir, portanto um registro de limiares incompleto.
3. Dois rótulos divergentes do registry atual: `crop_border_20pct` constava como `expected_match: true` e hoje é `false`, e `format_change_gif` constava como `image/gif` e hoje é `image/png`.

O manifest foi regerado com o código congelado, reaproveitando as imagens em cache (23 segundos, 0 erros).

As variantes não foram regeradas, e isso é deliberado. O `internal/dataset/transform/builders.go` usa um gerador pseudoaleatório global compartilhado entre os workers (`builders.go:700`), de modo que as famílias `noise_light` e `localized_recolor` **não são reproduzíveis** entre execuções paralelas — além de constituir uma corrida de dados. Reaproveitar os arquivos existentes preserva a reprodutibilidade do conjunto; regerá-los a destruiria. As únicas transformações afetadas pela regeneração foram as duas que mudaram de rótulo ou de formato.

Recomendação. `randSource` em `builders.go` deveria ser protegido por mutex ou instanciado por worker a partir de uma seed derivada do ID da base. Enquanto não for, a geração do dataset é irreprodutível em paralelo.

3.4 Passo 4 sem `docker compose down -v`

O roteiro manda apagar o volume do PostgreSQL para aplicar as 8 migrações do zero. Isso foi dispensado depois de verificar que o banco em execução já tinha as **8 tabelas** (`anchors`,

`api_keys`, `capture_nonces`, `certificates`, `devices`, `orgs`, `phash_bands`, `usage_counters`) e **todas as 19 colunas** que o código lê em `certificates`, mais a `phash_bands` completa. O teste `e2e` executa `TRUNCATE certificates CASCADE` por conta própria, que é todo o estado limpo de que precisa.

A razão para dispensar foi concreta: um `docker-compose.override.yml` local aponta o serviço `api` para a **Polygon mainnet com uma chave de ancoragem real**, e o `down -v` reiniciaria esse contêiner e destruiria um volume de minio sem relação com o experimento. O banco continha 1 certificado residual, truncado pelo teste.

4. Passo 1 — o dataset

4.1 Geração

```
go run -tags datasetgen ./cmd/datasetgen \  
  --source picsum --count 1000 --seed 42 --workers 4 \  
  --out testdata/generated
```

4.2 Metadados do manifest

campo	valor
<code>generator_version</code>	1.0.0
<code>run_id</code>	2026-10-02T190807-00000000
<code>seed</code>	42
<code>dataset_source</code>	picsum
<code>source_attribution</code>	Lorem Picsum — Unsplash License
<code>base_count</code>	993
<code>variants_per_base</code>	54
<code>sample_count</code>	53.622
<code>thresholds_snapshot</code>	idêntico aos limiares congelados
Tamanho em disco	7,3 GiB

O catálogo do Picsum tem 993 imagens com ID numérico, não 1.000. Portanto `--count 1000` seleciona todas as 993 disponíveis. O texto deve reportar 993 bases e 53.622 amostras, não "mil".

4.3 Tempos de execução observados

etapa	workers	duração	jobs
Regeração do manifest	4	24 s	53.622 arquivos
Certificação das 993 bases (simulação)	1	1 min 22 s	993
Passo 3, rodada de acurácia	4	2 h 34 min 09 s	54.615
Passo 3, rodada de latência	1	28 min 06 s	2.732
Passo 2, matriz par a par	1	1 h 10 min 12 s	53.277
Passo 4, end-to-end	4	3 h 50 min 49 s	53.622

Tempo total de medição: **8 h 03 min**, mais a geração do dataset. Os três níveis rodaram em sequência, nunca em paralelo.

A rodada de acurácia manteve 170 ms por amostra de forma estável do início ao fim, com **zero erros de infraestrutura** em 54.615 jobs.

4.4 Bases sem features ORB

Quatro das 993 bases não produzem nenhum keypoint ORB: `picsum_682`, `picsum_728`, `picsum_852`, `picsum_994`. São imagens de textura insuficiente.

Isto não é defeito do dataset: a certificação aceita tais imagens e grava um certificado **somente com pHash**, porque em `usecase.CertifyUseCase` a falha de extração é explicitamente não fatal. O experimento reproduz esse comportamento. A consequência é que essas bases não podem ser casadas visualmente, e suas 216 amostras (4×54) são decididas por `orb_extract_failed` ou pelo pré-filtro.

5. Mudanças de código antes do congelamento

5.1 Correção: decodificadores de BMP, TIFF e WebP no pHash

O defeito

A API aceita todo formato que o extractor baseado em OpenCV decodifica. O `internal/handler/upload.go` tem na allowlist `image/webp`, `image/bmp` e `image/tiff`, e o `decodeBGR` do extractor usa `gocv.IMDecode`, que trata os três sem problema.

O pHash seguia outro caminho. O `domain.PHash256` usa o `image.Decode` da biblioteca padrão de Go, que só conhece os formatos cujos pacotes foram registrados por import — e apenas `gif`, `jpeg` e `png` estavam. O resultado: um upload BMP, TIFF ou WebP produzia **pHash**

nulo, e `verify.go` respondia "imagem não decodificável" para uma imagem que o matcher trata perfeitamente.

Esse é o motivo pelo qual, antes da correção, `format_change_bmp` e `format_change_tiff` pontuavam **0% no pipeline e 100% par a par**: o par a par nunca consulta o pHash.

A correção

Registro dos decodificadores de `golang.org/x/image`:

```
- "golang.org/x/image/bmp"
- "golang.org/x/image/tiff"
- "golang.org/x/image/webp"
```

A versão foi fixada em `golang.org/x/image v0.34.0` porque a `@latest` (v0.46.0) exige Go ≥ 1.26 , acima do `go.mod` do projeto.

É correção de bug, não ajuste de limiar — mas está declarada aqui porque muda o resultado de duas famílias.

Efeito medido

transformação	pipeline antes	pipeline depois	par a par
<code>format_change_bmp</code>	0,000	0,990	0,995
<code>format_change_tiff</code>	0,000	0,990	0,995
<code>format_change_png</code>	0,990	0,990	0,995
<code>format_change_gif</code>	0,990	0,990	0,995

E, decisivamente: **nenhuma amostra do benchmark terminou em `phash_undecodable`**. A divergência entre os dois caminhos de decodificação foi eliminada.

Teste

`TestPHash256_DecodesEveryAcceptedFormat` fixa a cobertura de decodificadores, não a fidelidade de encoder: a asserção de distância de Hamming aplica-se apenas aos formatos sem perda (PNG, BMP, TIFF), porque o `gif.Encode` da biblioteca padrão aproxima para a paleta fixa Plan9 e o JPEG subamostra croma — asserções sobre esses dois mediriam o encoder, não o decodificador. O WebP, que `golang.org/x/image` decodifica mas não codifica, é coberto por `TestPHash256_WebPDecoderRegistered`, que alimenta um WebP truncado e distingue "formato desconhecido" (decodificador ausente) de um decodificador presente que rejeita carga malformada.

5.2 Caracterização: o resíduo de cor das rotações cardeais

A suspeita original

O roteiro registrava: "90° e 180° passaram em 35% e 270° em 90%. Os três são permutações de pixels sem perda, então há suspeita de erro no warp."

A investigação

A assimetria foi reproduzida em 12 bases: **50% / 25% / 83%**. A sonda instrumentada mostrou, em todos os casos:

medida	valor observado
Gate que falha	sempre ColorMax > 38
Inliers	1.479 a 1.915 (limiar: 20)
Cobertura	0,973 a 1,000 (limiar: 0,42)
Distância pHash	0 a 8 (limiar: 96)
ColorMean	0,58 a 4,14 (limiar: 8,0)

Ou seja, nem o pré-filtro, nem o estágio geométrico, nem o resíduo médio de cor ficam perto de reprovar. Apenas o **máximo por célula** estoura.

A primeira hipótese — vazamento do preenchimento de borda nas células parciais — foi **refutada**: as piores células estão no interior da imagem e totalmente cobertas pela máscara (`maskcov = 1,000`), não nas bordas.

O teste decisivo

Substituindo a homografia estimada por RANSAC pela **rotação analítica exata** (uma matriz 3×3 construída a partir da permutação de pixels que o `gocv.Rotate` aplica), medido sobre as mesmas amostras:

base	rotação	ColorMax com RANSAC	ColorMax exato
picsum_0	90°	45,8	3,9
picsum_0	180°	73,1	3,9
picsum_0	270°	45,2	3,0
picsum_1	90°	49,3	3,0
picsum_1	180°	82,9	3,2
picsum_1001	180°	86,5	5,7
picsum_1003	180°	67,3	3,5

Com a homografia exata, **todas as rotações passam**, com ColorMax entre 2,0 e 6,3 — contra 23 a 86 sob RANSAC.

A conclusão

O gate de cor está correto e o pré-filtro está correto. **O limite é a precisão do alinhamento geométrico.** O grid de 128×128 sobre uma referência de 800×600 produz células de cerca de **6×5 pixels**: um erro sub-pixel na homografia desloca a média de uma célula de alta frequência em dezenas de unidades LAB, e o gate por célula lê isso como uma edição localizada.

Não é erro de warp no gerador de dataset. É uma sensibilidade intrínseca do desenho do gate à resolução do grid.

Duas correções medidas e rejeitadas

estratégia	resultado em 314 pares
Reajustar a homografia nos inliers do RANSAC por mínimos quadrados	indistinguível do baseline em todos os 314 pares; o OpenCV já executa refinamento no conjunto de consenso
Apertar o limiar de reprojeção do RANSAC de 5,0 para 2,0	rotações melhoram (90°: 40→67%, 180°: 20→73%, JPEG q10: 73→93%), escala e corte pioram (downscale 0,5x: 33→20%, upscale 2x: 80→53%, crop 10%: 93→80%). Total: 249 → 252 de 314

Nenhuma linha de negativo mudou em nenhuma das duas estratégias — a precisão é insensível a essa escolha. Como nenhuma é ganho líquido, e qualquer alternativa mais ambiciosa exigiria mexer numa constante congelada ou dispor da imagem de referência — que a arquitetura deliberadamente **não armazena**, guardando apenas o grid LAB de 128×128 —, o matcher **não foi alterado**.

O diagnóstico ficou fixado em `TestCardinalRotation_ResidualIsAlignmentBound`, que afirma o que é robusto: com o warp exato todos os gates passam, e com o warp estimado os gates de inlier e cobertura continuam folgados, de modo que qualquer falha futura em rotação é atribuível à precisão do alinhamento.

Recomendação. Para melhorar rotações sem relaxar limiar, seria preciso refinamento sub-pixel do alinhamento, o que exigiria armazenar uma miniatura de referência em vez de apenas as médias por célula. É uma decisão de arquitetura, com custo de armazenamento e de privacidade, não um ajuste.

6. Instrumentação: como o experimento mede

6.1 Fidelidade ao caminho de produção

O `TestTCC_PipelineSimulation` **chama o** `usecase.VerifyUseCase.Execute real`, alcançando-o pela porta `observability.Recorder`, em vez de reimplementar o pipeline. A consequência é que os limites de estágio e a lógica de decisão medidos são **os de produção**, e os tempos por estágio vêm da própria instrumentação da aplicação.

6.2 O repositório em memória

O `tccMemRepo` implementa `usecase.CertificateRepository` reproduzindo a semântica do `repository.PostgresCertificateRepo`:

- `FindByHash` é consulta exata por hash de conteúdo.
- `FindCandidatesByPHashes` resolve o pré-filtro LSH do mesmo modo: cada um dos 32 bytes de banda de cada uma das 4 rotações candidatas sonda um índice (`band_idx`, `band_value`); os certificados que colidem são repontuados com a distância de Hamming exata de 256 bits; os acima de `MaxPHashDistance` são descartados; os restantes são ordenados por distância e truncados em `topK`.

6.3 Diferenças conhecidas em relação à API real

1. **Ordem de empate.** O PostgreSQL deixa indefinida a ordem entre candidatos empatados em distância; a simulação desempata pelo hash de conteúdo, para ser reprodutível. O passo 4 é o que estabelece se isso importa.
2. **verifyTopK não é exportado.** A cópia no teste é conferida **em cada amostra** contra a contagem de candidatos que o use case reportou: se a lista ordenada era maior que a janela, a contagem relatada tem de ser exatamente o tamanho da janela. A cópia não pode derivar em silêncio.

6.4 O controle negativo acrescentado

O manifest não consegue expressar o controle que mais importa para um certificador. O `different_image` devolve **os bytes do par intactos**, e o par é ele mesmo certificado — então o SHA-256 resolve antes de qualquer comparação visual. Ele nunca exercita o caminho que decidiria uma falsificação. Os dados confirmam: todas as 993 amostras `different_image` foram decididas em `sha256`.

O controle `uncertified_image` consulta os bytes de uma base contra uma **view** do repositório com aquele certificado escondido. O SHA-256 então não resolve, e a correspondência visual roda contra os outros 992 certificados **sem resposta correta disponível**. É um controle por base, 993 no total.

A view esconde o certificado em vez de removê-lo do banco, para o controle rodar em paralelo sem corrida com as demais amostras.

6.5 Colunas registradas por amostra

O CSV tem 31 colunas por amostra: identificação e rótulo (8), veredito do pipeline (4), veredito par a par e métricas do matcher (7), diagnósticos do pré-filtro (5) e latência por estágio (7).

As colunas de pré-filtro são o que atribui a diferença entre os dois vereditos: `prefilter_has_correct`, `prefilter_correct_rank`, `prefilter_ranked_total`, `prefilter_candidates` e `phash_hamming`.

6.6 Comandos executados

```
# Rodada de acurácia: todas as amostras, 4 workers
TCC_OUT=results/tcc_results.csv TCC_WORKERS=4 \
go test -tags integration -count=1 -timeout 0 -v \
-run TestTCC_PipelineSimulation ./tests/feature/

# Rodada de latência: 1 em cada 20 amostras, uma thread
TCC_OUT=results/tcc_latency.csv TCC_WORKERS=1 TCC_STRIDE=20 \
go test -tags integration -count=1 -timeout 0 -v \
-run TestTCC_PipelineSimulation ./tests/feature/

# Resumos com intervalos de Wilson a 95%
python3 scripts/tcc_summary.py results/tcc_results.csv --out results/resumo_tcc.md
python3 scripts/tcc_summary.py results/tcc_latency.csv --out results/resumo_latencia.md
```

7. Passo 3 — resultados de acurácia

7.1 Pipeline completo

Uma amostra conta como acerto apenas quando o certificado devolvido é **o da base correta**. As linhas excluem os 993 controles `uncertified_image`, reportados na seção 7.8.

estrato	n	TP	FP	FN	TN	precisão [IC 95%]	recall [IC 95%]	especificidade [IC 95%]	acurácia [IC 95%]
Alta confiança	16.881	8.783	3	154	7.941	1,000 [0,999, 1,000]	0,983 [0,980, 0,985]	1,000 [0,999, 1,000]	0,991 [0,989, 0,992]
Limítrofe	36.741	15.766	1.884	10.052	9.039	0,893 [0,889, 0,898]	0,611 [0,605, 0,617]	0,828 [0,820, 0,834]	0,675 [0,670, 0,680]
Todas	53.622	24.549	1.887	10.206	16.980	0,929 [0,925, 0,932]	0,706 [0,702, 0,711]	0,900 [0,896, 0,904]	0,774 [0,771, 0,778]

7.2 Par a par, para leitura lado a lado

Veredito do matcher isolado: ORB, RANSAC, resíduo de cor e cobertura contra a base correta, sem busca. É o teto que o pipeline persegue.

estrato	n	TP	FP	FN	TN	precisão [IC 95%]	recall [IC 95%]	especificidade [IC 95%]	acurácia [IC 95%]
Alta confiança	16.748	8.793	4	108	7.843	1,000 [0,999, 1,000]	0,988 [0,985, 0,990]	0,999 [0,999, 1,000]	0,993 [0,992, 0,994]
Limítrofe	36.529	19.503	1.889	6.167	8.970	0,912 [0,908, 0,915]	0,760 [0,754, 0,765]	0,826 [0,819, 0,833]	0,779 [0,775, 0,784]
Todas	53.277	28.296	1.893	6.275	16.813	0,937 [0,935, 0,940]	0,818 [0,814, 0,823]	0,899 [0,894, 0,903]	0,847 [0,844, 0,850]

O `n` par a par é menor — 53.277 contra 53.622 — porque em 345 amostras a extração ORB da variante falhou, e sem assinatura não há comparação par a par a fazer. A API, nesse caso, responde `no_match`.

A leitura conjunta das duas tabelas é o resultado central do capítulo. No estrato de alta confiança os dois níveis são praticamente idênticos: o pipeline entrega 0,983 de recall contra 0,988 do matcher isolado, uma perda de 0,5 ponto. No estrato limítrofe a diferença explode: 0,611 contra 0,760, **quase 15 pontos de recall perdidos na busca**. A seção 7.6 atribui essa perda.

7.3 Robustez do resultado a duas objeções

Sem as 100 bases que definiram o estrato limítrofe

Os comentários do registry citam taxas observadas em 100 imagens do Picsum, de modo que o estrato "limítrofe" foi definido **depois** de ver o sistema. Essas 100 bases estão entre as 993.

Os IDs foram reconstruídos: o `cmd/datasetgen` usa `--count 100 --seed 42` por padrão para a fonte `picsum`, o `source.Picsum.List` embaralha o catálogo travado com essa seed antes de cortar, e o embaralhamento é estável no prefixo. Verificou-se que a lista coincide exatamente com os 100 primeiros `base_image_id` distintos na ordem do `manifest.json`. É **reconstrução por inferência**, não um registro daquela execução, e isso deve ser declarado.

estrato	n	TP	FP	FN	TN	precisão	recall
Alta confiança	15.181	7.899	2	138	7.142	1,000 [0,999, 1,000]	0,983 [0,980, 0,985]
Limítrofe	33.041	14.210	1.708	9.008	8.115	0,893 [0,888, 0,897]	0,612 [0,606, 0,618]
Todas	48.222	22.109	1.710	9.146	15.257	0,928 [0,925, 0,931]	0,707 [0,702, 0,712]

Idêntico até a terceira decimal. O risco metodológico do estrato definido a posteriori não se materializou.

Sem o par de bases duplicadas

A seção 10 detalha a descoberta de que `picsum_128` e `picsum_456` são a mesma fotografia. Excluindo ambas:

estrato	n	TP	FP	FN	TN	precisão	recall
Alta confiança	16.847	8.774	3	145	7.925	1,000 [0,999, 1,000]	0,984 [0,981, 0,986]
Limítrofe	36.667	15.749	1.883	10.017	9.018	0,893 [0,889, 0,898]	0,611 [0,605, 0,617]
Todas	53.514	24.523	1.886	10.162	16.943	0,929 [0,925, 0,932]	0,707 [0,702, 0,712]

O recall de alta confiança sobe de 0,983 para 0,984; a precisão permanece 1,000.

7.4 Análise dos falsos positivos

Alta confiança: três eventos, todos `sepia`

amostra	inliers			cobertura
<code>picsum_1074_sepia</code>	1.831	7,660	34,608	1,000
<code>picsum_556_sepia</code>	787	6,012	35,701	1,000
<code>picsum_811_sepia</code>	1.816	7,521	33,991	1,000

Os três passaram porque o resíduo ficou **logo abaixo dos dois limiares de cor**: média entre 6,0 e 7,7 contra o limite de 8,0, e máximo por célula entre 34,0 e 35,7 contra o limite de 38,0. São imagens cuja paleta original já é próxima do sépia, de modo que a transformação quase não move o resíduo LAB. O restante da família acertou: `sepia` fecha em 0,997 (990 de 993).

Com 3 eventos em 8.786 predições positivas, a taxa de falso positivo de alta confiança é 0,00034, com limite superior de Wilson em torno de 0,1%.

Limítrofe: 1.884 eventos, concentrados em saturação e matiz

transformação	FP	n	taxa
<code>saturation_boost_1.5x</code>	650	993	0,655
<code>localized_recolor</code>	284	993	0,286
<code>hue_shift_30deg</code>	238	993	0,240
<code>grayscale</code>	217	993	0,219
<code>saturation_boost_2.0x</code>	187	993	0,188
<code>hue_shift_60deg</code>	125	993	0,126
<code>hue_shift_120deg</code>	99	993	0,100
<code>hue_shift_180deg</code>	84	993	0,085

O padrão é coerente com a documentação do próprio limiar: edições globais de cor de baixa amplitude, sobre imagens de baixa saturação, não movem o resíduo LAB acima de 8,0, porque os canais a/b já estão próximos de zero. `saturation_boost_1.5x` é o pior caso com 65,5% de falso positivo — um aumento de 50% na saturação de uma foto já dessaturada é quase imperceptível no espaço LAB.

Note a inversão de monotonicidade: **1,5× falha mais que 2,0×** (0,655 contra 0,188), e `hue_shift_30deg` falha mais que 180° (0,240 contra 0,085). Quanto menor a edição, mais o sistema a considera o mesmo conteúdo — o que é o comportamento desejado de um verificador de identidade e sugere que o rótulo "quebra identidade" para amplitudes pequenas é discutível como verdade fundamental.

7.5 Análise dos falsos negativos

Alta confiança: 154 eventos

transformação	FN
<code>jpeg_recompress_q20</code>	30
<code>jpeg_recompress_q30</code>	21
<code>noise_sigma10</code>	20
<code>whatsapp_like_960px_q40</code>	17
<code>jpeg_recompress_q50</code>	15
<code>noise_sigma5</code>	15
<code>jpeg_recompress_q70</code>	13
<code>sharpen_light</code>	12
<code>jpeg_recompress_q90</code>	11

Por estágio decisor: `no_match` 113, `orb_extract_failed` 30, `visual_match` 9, `prefilter_empty` 2.

Os 30 de `orb_extract_failed` são as variantes das quatro bases sem features. Os 9 de `visual_match` são casos do par duplicado da seção 10 — casaram, mas com o certificado da gêmea. Os 113 de `no_match` são o resíduo genuíno: imagens em que a recompressão ou o ruído baixou os inliers abaixo de 20 ou subiu o resíduo acima do limiar.

Que `jpeg_recompress_q90` tenha 11 falsos negativos enquanto `q20` tem 30 mostra que a causa não é a qualidade da compressão, e sim o conteúdo: imagens de pouca textura perdem inliers independentemente do nível.

7.6 O pré-filtro — a origem da perda de recall

Esta é a seção que explica a diferença entre as tabelas 7.1 e 7.2.

Medido apenas nas 34.755 amostras com rótulo positivo, onde existe um certificado correto a encontrar:

medida	eventos	taxa [IC 95%]
Certificado correto chegou à lista de candidatos	30.549	0,879 [0,876, 0,882]
Certificado correto dentro da janela top-64	30.549	0,879 [0,876, 0,882]
Certificado correto em primeiro lugar	30.454	0,876 [0,873, 0,880]

As duas primeiras linhas são idênticas. A janela top-64 nunca descartou um certificado que havia sobrevivido ao corte de Hamming. A distribuição de candidatos confirma por quê:

distribuição	mediana	p95	máximo
Candidatos avaliados por consulta	1	3	12
Distância pHash até o certificado correto	10	116	138

Achado. Com no máximo 12 candidatos observados em 54.615 consultas, a janela de 64 está ordens de magnitude acima do necessário. O comentário em [verify.go](#) já suspeitava disso; os dados confirmam. O `verifyTopK` poderia ser reduzido substancialmente sem perda de recall, e isso reduziria o pior caso de latência de uma consulta negativa.

As duas causas de perda, que têm consequências opostas

causa	n	fração das positivas [IC 95%]
Distância pHash acima de 96	4.125	0,119 [0,115, 0,122]
Dentro de 96, mas sem colisão de banda	81	0,002 [0,002, 0,003]

A distinção importa porque as duas apontam para partes diferentes do desenho. A sonda por bandas só recupera um certificado quando pelo menos **um dos 32 bytes** do pHash coincide exatamente; um certificado pode estar dentro do limiar de Hamming e ainda assim nunca ser avaliado. Esses 81 casos são um limite de recall do esquema de bandas, que nem um top-K maior nem um limiar de Hamming mais folgado recuperariam. Mas são apenas 0,2% — **o esquema de bandas é eficiente**. A perda real, 11,9%, é o limiar de Hamming.

Os falsos negativos recuperáveis

Em **4.206** amostras positivas o certificado correto nunca chegou à correspondência visual. Dessas, **3.721 teriam casado par a par**: são falsos negativos atribuíveis inteiramente ao pré-filtro.

transformação	perdas	das quais casariam par a par
rotate_32deg	985	836
crop_border_15pct	962	886
crop_border_10pct	891	814
rotate_10deg	884	768
crop_border_5pct	251	224
rotate_5deg	227	192
noise_sigma10	3	1
downscale_256px	1	0
jpeg_recompress_q10	1	0
downscale_160px	1	0

A perda está **inteiramente concentrada em rotação de ângulo pequeno e corte de borda**. Ambas deslocam o conteúdo no quadro, o que desloca a DCT de baixa frequência que o pHash amostra, enquanto preservam todas as feições locais que o ORB usa. É precisamente o modo de falha que um pré-filtro perceptual global tem e um matcher de feições locais não tem.

A limitação estrutural que isso revela

O pHash **não distingue uma edição geométrica que preserva identidade de uma que a destrói**. Compare as taxas de recuperação do certificado correto:

transformação	rótulo	pré-filtro recuperou
crop_border_15pct	match	0,031
crop_border_20pct	reject	0,017
heavy_crop_40pct	reject	0,017

crop_border_15pct deveria casar e crop_border_20pct não, mas o pré-filtro trata as duas quase igualmente. Isso não é inconsistência do sistema: é o pré-filtro sendo coerente com sua própria métrica, enquanto o rótulo traça uma linha que o pHash não enxerga. O próprio comentário em feature-signature.go reconhece que crop_border_20pct e heavy_crop_40pct são **a mesma imagem pixel a pixel**, com rótulos opostos na taxonomia.

7.7 Estágio que decidiu

Sobre os 54.615 jobs, incluindo controles:

estágio	n	fração
visual_match	26.465	0,485
no_match	19.988	0,366
prefilter_empty	6.851	0,125
sha256	993	0,018
orb_extract_failed	318	0,006
phash_undecodable	0	0,000

Observações:

- **phash_undecodable em zero** é a confirmação da correção da seção 5.1.
- Os 993 de sha256 são exatamente as amostras different_image, o que demonstra o ponto da seção 6.4: esse controle nunca exercita a correspondência visual.
- prefilter_empty em 12,5% é o pré-filtro corretamente não encontrando nada para edições que destroem o pHash — e incorretamente, para rotações pequenas e cortes de borda.
- orb_extract_failed concentra-se onde se espera: heavy_crop_60pct (43), heavy_crop_50pct (26), downscale_160px (22), crop_border_20pct (14), heavy_crop_40pct (14).

Um caso notável: color_invert foi decidido por prefilter_empty em 676 das 993 amostras. A inversão fotográfica aproximadamente complementa o pHash, de modo que nenhuma banda colide. O pré-filtro rejeita a inversão sem precisar do matcher — correto, e de graça.

7.8 Controles negativos

controle	n	eventos	taxa [IC 95%]
Imagem nunca certificada casou com algum certificado	993	2	0,002 [0,001, 0,007]
Atribuída a outro certificado por correspondência visual	52.629	27	0,001 [0,000, 0,001]
Resolvida pelo SHA-256 para o certificado do próprio par (correto por construção)	53.622	993	0,019

As duas primeiras linhas têm a mesma causa única, detalhada na seção 10: o par de bases duplicadas. Descontando-o:

- **Controle de imagem não certificada: 0 acertos em 991 tentativas válidas**, com limite superior de Wilson de **0,39%** a 95%.
- **Atribuições genuinamente erradas: zero.**

O texto deve reportar o limite superior, não "nunca erra". Zero eventos em 991 tentativas é compatível com uma taxa real de até 0,39%.

A terceira linha merece destaque metodológico: ela **não é erro**. Uma amostra `different_image` carrega os bytes do par intactos, e o par é certificado, então devolver o certificado do par é a resposta verdadeira. Contá-la como "atribuição errada" inflaria o número de erros em 36 vezes. Esse era um defeito do script de resumo, corrigido (seção 11.2).

7.9 Tabela completa por transformação

Colunas: `pipe` é o acerto do pipeline completo, `pair` o acerto par a par, `gap` a diferença (positiva quando o pré-filtro custou recall), `pf` a fração em que o certificado correto chegou aos candidatos. $n = 993$ em todas.

transformação	família	estrato	rótulo	pipe	pair	gap	pf
brightness_minus10pct	brightness	limítrofe	match	0,951	0,956	+0,005	1,000
brightness_minus5pct	brightness	limítrofe	match	0,987	0,992	+0,005	1,000
brightness_plus10pct	brightness	limítrofe	match	0,945	0,949	+0,005	1,000
brightness_plus5pct	brightness	limítrofe	match	0,988	0,993	+0,005	1,000
color_invert	color_invert	alta	reject	1,000	1,000	+0,000	0,002
content_overlay_10pct	content_overlay	limítrofe	reject	1,000	1,000	+0,000	0,996
content_overlay_15pct	content_overlay	alta	reject	1,000	1,000	+0,000	0,992
content_overlay_20pct	content_overlay	alta	reject	1,000	1,000	+0,000	0,991
content_overlay_30pct	content_overlay	alta	reject	1,000	1,000	+0,000	0,991
crop_border_5pct	crop_border	limítrofe	match	0,684	0,914	+0,230	0,747
crop_border_10pct	crop_border	limítrofe	match	0,091	0,917	+0,826	0,103
crop_border_15pct	crop_border	limítrofe	match	0,029	0,928	+0,899	0,031
crop_border_20pct	crop_border	limítrofe	reject	1,000	0,998	−0,002	0,017
different_image	different_image	alta	reject	1,000	1,000	+0,000	0,000
downscale_0.33x	downscale	limítrofe	match	0,044	0,045	+0,001	1,000
downscale_0.5x	downscale	limítrofe	match	0,401	0,404	+0,003	1,000
downscale_0.75x	downscale	limítrofe	match	0,841	0,846	+0,005	1,000
downscale_160px	downscale	limítrofe	match	0,000	0,000	+0,000	0,999
downscale_256px	downscale	limítrofe	match	0,042	0,043	+0,001	0,999
format_change_bmp	format_change	limítrofe	match	0,990	0,995	+0,005	1,000
format_change_gif	format_change	limítrofe	match	0,990	0,995	+0,005	1,000
format_change_png	format_change	limítrofe	match	0,990	0,995	+0,005	1,000
format_change_tiff	format_change	limítrofe	match	0,990	0,995	+0,005	1,000
grayscale	grayscale	limítrofe	reject	0,781	0,781	−0,001	1,000
heavy_crop_40pct	heavy_crop	limítrofe	reject	1,000	0,998	−0,002	0,017
heavy_crop_50pct	heavy_crop	alta	reject	1,000	0,999	−0,001	0,010
heavy_crop_60pct	heavy_crop	alta	reject	1,000	1,000	+0,000	0,005
hue_shift_30deg	hue_shift	limítrofe	reject	0,760	0,759	−0,001	1,000
hue_shift_60deg	hue_shift	limítrofe	reject	0,874	0,874	−0,001	1,000
hue_shift_120deg	hue_shift	limítrofe	reject	0,900	0,900	−0,000	1,000
hue_shift_180deg	hue_shift	limítrofe	reject	0,915	0,915	−0,000	0,997
jpeg_recompress_q10	jpeg_recompress	limítrofe	match	0,916	0,921	+0,005	0,999
jpeg_recompress_q20	jpeg_recompress	alta	match	0,970	0,975	+0,005	1,000
jpeg_recompress_q30	jpeg_recompress	alta	match	0,979	0,984	+0,005	1,000

transformação	família	estrato	rótulo	pipe	pair	gap	pf
jpeg_recompress_q50	jpeg_recompress	alta	match	0,985	0,990	+0,005	1,000
jpeg_recompress_q70	jpeg_recompress	alta	match	0,987	0,992	+0,005	1,000
jpeg_recompress_q90	jpeg_recompress	alta	match	0,989	0,994	+0,005	1,000
localized_recolor	localized_recolor	limítrofe	reject	0,714	0,713	−0,001	1,000
noise_sigma5	noise_light	alta	match	0,985	0,990	+0,005	1,000
noise_sigma10	noise_light	alta	match	0,980	0,986	+0,006	0,997
p3_as_srgb_q70	p3_as_srgb	limítrofe	match	0,840	0,844	+0,004	1,000
rotate_90	rotate_cardinal	limítrofe	match	0,397	0,399	+0,003	1,000
rotate_180	rotate_cardinal	limítrofe	match	0,437	0,440	+0,003	1,000
rotate_270	rotate_cardinal	limítrofe	match	0,912	0,917	+0,005	1,000
rotate_5deg	rotate_small	limítrofe	match	0,680	0,878	+0,198	0,771
rotate_10deg	rotate_small	limítrofe	match	0,092	0,869	+0,777	0,110
rotate_32deg	rotate_small	limítrofe	match	0,005	0,850	+0,845	0,008
saturation_boost_1.5x	saturation_boost	limítrofe	reject	0,345	0,342	−0,004	1,000
saturation_boost_2.0x	saturation_boost	limítrofe	reject	0,812	0,811	−0,001	1,000
sepia	sepia	alta	reject	0,997	0,997	−0,000	0,999
sharpen_light	sharpen	alta	match	0,988	0,993	+0,005	1,000
upscale_1.5x	upscale	limítrofe	match	0,809	0,813	+0,004	1,000
upscale_2.0x	upscale	limítrofe	match	0,828	0,831	+0,003	1,000
whatsapp_like_960px_q40	whatsapp_like	alta	match	0,983	0,988	+0,005	1,000

Como há cerca de 993 amostras por transformação, a meia-largura do intervalo de 95% fica em no máximo 3 pontos percentuais. **Diferenças menores que isso entre famílias não devem ser interpretadas.**

Leituras notáveis da tabela

- **Seis transformações com gap grande** — `crop_border_5/10/15pct` e `rotate_5/10/32deg` — são onde o pré-filtro custa recall. Em todas as outras 48 o `gap` fica em $\pm 0,006$, ou seja, o pipeline iguala o matcher isolado.
- **downscale_160px falha completamente (0,000)** e o pré-filtro **recupera** o certificado em 99,9% dos casos. Logo a falha é do matcher, não da busca: a razão de escala excede o alcance confiável do ORB, e 22 amostras nem produzem features. `downscale_0.33x` (0,044) e `downscale_256px` (0,042) têm a mesma causa.
- **Os gap negativos são minúsculos** (máximo −0,004) e aparecem em famílias de rejeição. São casos em que o pré-filtro ajudou, descartando o candidato antes de o matcher ter chance de errar.

- **A escada de recompressão JPEG é monotônica e apertada:** 0,989 em q90 descendo a 0,916 em q10. É o comportamento esperado e o estrato de alta confiança cobre q20 a q90.
- **A assimetria das rotações cardeais** (0,397 / 0,437 / 0,912) persiste com `pf = 1,000` nas três, confirmando pela terceira via que o pré-filtro não é a causa — é o gate `ColorMax`, conforme a seção 5.2.

8. Passo 3 — resultados de latência

Rodada em **uma thread** (`TCC_WORKERS=1`), 1 em cada 20 amostras, 2.732 jobs em 28 min 06 s, no AMD EPYC-Rome de 4 vCPU descrito na seção 2.2. Os valores medem o custo do pipeline sem disputa de CPU.

estágio	n	mediana (ms)	p95 (ms)	máximo (ms)
SHA-256	2.732	0,1	0,4	1,5
Consulta exata por hash	2.732	0,0	0,0	0,0
pHash (4 rotações)	2.732	218,0	289,3	687,7
Extração ORB	2.732	26,1	42,3	80,2
Pré-filtro LSH	2.732	0,5	0,6	1,3
Correspondência visual	2.732	50,4	93,1	264,4
Verificação completa	2.732	289,5	410,8	883,0

veredito	n	mediana total (ms)	p95 total (ms)
Casou	1.358	309,8	500,8
Não casou	1.374	265,3	401,1

8.1 O achado: o pHash domina a latência

O cálculo do pHash consome 218 ms dos 289,5 ms de mediana, ou 75% do custo total de uma verificação. Para comparação, a extração ORB — que envolve detecção de 2.000 keypoints e descritores binários — leva 26,1 ms, oito vezes menos.

A causa é a implementação de `dct2D` em `internal/domain/perceptual_hash.go`: uma DCT 2D direta, de quatro laços encaixados, que para $N = 32$ executa $N^4 = 1.048.576$ iterações, cada uma com **duas chamadas a `math.Cos`**. O `PHash256Variants` faz isso quatro vezes, uma por rotação, totalizando cerca de **8,4 milhões de avaliações de cosseno por verificação**.

Recomendação. Duas otimizações independentes, ambas sem alterar o valor do hash: (a) pré-computar a matriz de cossenos de 32×32 uma única vez, eliminando as 8,4 milhões de chamadas trigonométricas; (b) usar a separabilidade da DCT 2D, reduzindo o custo de $O(N^4)$ para $O(N^3)$. Combinadas, a expectativa é uma redução de uma a duas ordens de magnitude no estágio, levando a mediana da verificação completa de ~ 290 ms para a faixa de 80–100 ms. Como o hash resultante é idêntico, isso não afeta nenhum resultado de acurácia deste benchmark.

Vale registrar que o pré-filtro LSH, que é o estágio arquiteturalmente interessante, custa **0,5 ms** — desprezível. A escolha de desenho de indexar bandas de pHash num índice composto está vindicada no eixo de desempenho; o problema está em calcular o pHash, não em buscar por ele.

8.2 Consultas que casam versus que não casam

Uma consulta que encontra sua correspondência para no primeiro candidato; uma que não encontra paga a comparação contra todos os candidatos. A diferença observada é modesta — 309,8 ms contra 265,3 ms de mediana — e o sinal é **invertido** em relação ao esperado: as que não casam são mais rápidas.

A explicação está na seção 7.6: a mediana de candidatos por consulta é 1. Uma consulta negativa normalmente tem **zero ou um** candidato a comparar, então ela economiza o estágio de correspondência visual em vez de pagá-lo 64 vezes. O pior caso teórico que motivou `verifyTopK = 64` praticamente não ocorre neste banco de 993 certificados.

9. Passo 2 — verificação cruzada par a par

O passo 2 roda a ferramenta de matriz que já existia no repositório, `TestDataset_FullMatrix`, em uma única thread. Ele não contribui números novos: os números par a par oficiais saem da coluna `pair_matched` do passo 3. A função dele é **confirmar que a simulação concorda com a ferramenta original**.

item	valor
Duração	1 h 10 min 12 s (4.212 s)
Threads	1
Bases avaliadas	989 de 993
Bases puladas (sem features ORB)	4
Amostras puladas com elas	216
Erros de infraestrutura por amostra	129
Amostras efetivamente avaliadas	53.277
Veredito do teste	FAIL , conforme previsto pelo protocolo

9.1 Resultado

estrato	TP	FP	FN	TN	precisão	recall	F1
Todas	28.296	1.893	6.275	16.813	0,937	0,818	0,874
Alta confiança	8.793	4	108	7.843	0,9995	0,9879	0,9937

9.2 A verificação cruzada: concordância exata

Os quatro totais do `matrix_report.json` são **idênticos** à tabela "Par a par" do passo 3, em ambos os estratos:

estrato	fonte	TP	FP	FN	TN
Todas	<code>TestDataset_FullMatrix</code> (passo 2)	28.296	1.893	6.275	16.813
Todas	coluna <code>pair_matched</code> (passo 3)	28.296	1.893	6.275	16.813
Alta confiança	<code>TestDataset_FullMatrix</code> (passo 2)	8.793	4	108	7.843
Alta confiança	coluna <code>pair_matched</code> (passo 3)	8.793	4	108	7.843

E o fecho de contas das amostras também é exato. O passo 2 pulou 216 amostras (as das 4 bases sem features) e teve 129 erros de extração em variantes, totalizando 345 amostras sem veredito par a par. Como $53.622 - 345 = \mathbf{53.277}$, e esse é precisamente o `n` da tabela par a par do passo 3, as **mesmas** 345 amostras ficaram sem veredito nos dois caminhos independentes.

9.3 Por que o teste termina em FAIL

O `FAIL` é esperado e não indica problema. O `runManifestEval` conta uma falha de extração ORB numa **variante** como erro de infraestrutura e, ao final, chama `t.Fatalf`. O `report.json` é gravado **antes** dessa falha, então o artefato está completo.

Os 129 erros concentram-se exatamente onde a geometria destrói a textura:

transformação	erros
heavy_crop_60pct	39
heavy_crop_50pct	22
downscale_160px	18
heavy_crop_40pct	10
crop_border_20pct	10
downscale_256px	8
downscale_0.33x	8
different_image	4
crop_border_15pct	3
crop_border_10pct	3
downscale_0.5x	2
downscale_0.75x	1
outras	1

Vale notar a diferença de contabilidade entre os níveis: o `report.json` **exclui** essas 129 amostras da matriz, enquanto a API responde `no_match` para elas — e é assim que o pipeline do passo 3 as contabiliza, como falso negativo quando o rótulo era `match`. Os 4 erros em `different_image` são variantes cujo par não produz features; no pipeline completo, essas mesmas amostras são resolvidas pelo SHA-256 antes de qualquer extração.

Esta é a razão pela qual as tabelas 7.1 e 7.2 têm `n` diferentes (53.622 contra 53.277), e por que a comparação entre os dois níveis deve ser lida por taxa, não por contagem absoluta.

9.4 O que as três concordâncias estabelecem, em conjunto

O benchmark produziu três medições independentes que se validam mutuamente:

comparação	resultado
Par a par: ferramenta original (passo 2) × coluna da simulação (passo 3)	idêntico , 4 totais em 2 estratos
Pipeline: simulação em memória (passo 3) × e2e com PostgreSQL (passo 4)	idêntico , 4 totais em 3 estratos e nas 21 famílias
Contagem de amostras sem veredito par a par	idêntica , 345 em dois caminhos independentes

Nenhuma das três admitia folga no protocolo além de 1 ponto percentual, e as três deram divergência zero. Isso autoriza usar o CSV do passo 3 — com seus diagnósticos de pré-filtro, estágio decisor e latência por estágio — como fonte dos números do capítulo, em vez de apenas as matrizes agregadas.

10. Achado do dataset: duas bases são a mesma fotografia

10.1 Como apareceu

A rodada de acurácia acusou dois números anômalos: **27 amostras atribuídas a "outro certificado"** por correspondência visual, e **2 acertos no controle** `uncertified_image`, que por construção não deveria ter nenhum.

A primeira pista foi a distribuição: as 27 apareciam como **exatamente uma por transformação**, em 27 transformações diferentes. Erros independentes não se distribuem assim; isso indicava uma única base de origem. Confirmado: todas as 27 vêm de `picsum_456`.

10.2 A evidência

medida	valor
Distância pHash entre <code>picsum_456</code> e <code>picsum_128</code>	0
Inliers do matcher entre as duas	2.000 (o teto de <code>orbFeatures</code>)
<code>ColorMean</code>	0,44
<code>ColorMax</code>	0,9
Cobertura	1,000
Pares de bases dentro de Hamming 32, em 492.528 pares	1 — exatamente este
Duplicatas byte a byte entre as 993 bases	0

O segundo vizinho mais próximo de ambas está a Hamming 102, com 5 inliers e `ColorMean` 56 — ou seja, não há gradação: existe um par idêntico e nada mais perto que o ruído.

As duas são **a mesma fotografia servida sob dois IDs do Lorem Picsum**, com encodings JPEG distintos (daí não serem byte-idênticas).

10.3 Consequências para o relato

- As 27 atribuições a "outro certificado" são variantes da `picsum_456` casando com o certificado da gêmea idêntica. É a **resposta correta**: as imagens são o mesmo conteúdo. O critério de acerto do experimento exige o certificado da base de origem, então são contadas como erro sem o ser. **Zero atribuições genuinamente erradas.**
- Os 2 acertos do controle são o mesmo par: esconder o certificado da `picsum_456` ainda deixa no banco o certificado idêntico da `picsum_128`, que casa legitimamente. O controle tem **2 tentativas contaminadas em 993**; nas 991 válidas o resultado é **0 acertos**, limite superior 0,39%.

- O efeito na manchete é pequeno: recall de alta confiança 0,983 → 0,984, precisão inalterada em 1,000 (seção 7.3).

10.4 A limitação metodológica a declarar

Um catálogo de imagens públicas pode servir o mesmo conteúdo sob identificadores diferentes. Um benchmark que trata cada identificador como uma identidade distinta precisa verificar isso antes de chamar uma colisão de erro — e a verificação é barata: distância pHash par a par entre as bases, que para 993 bases são 492.528 comparações de 32 bytes, concluídas em segundos.

Recomenda-se incluir essa checagem no gerador de dataset, emitindo aviso quando duas bases ficarem abaixo de um limiar de proximidade.

11. Defeitos encontrados nas ferramentas de medição

Dois defeitos foram encontrados **lendo os resultados**, e corrigidos no commit `5a36615`, posterior à tag. Nenhum toca o sistema sob teste: o `TestDataset_FullMatrix` é a verificação cruzada opcional do passo 2, e o `tcc_summary.py` apenas formata um CSV.

11.1 `TestDataset_FullMatrix` abortava na primeira base sem features

O `runManifestEval` chamava `t.Fatalf` no momento em que uma **base** não produzisse features ORB. Como 4 das 993 bases são assim, a primeira tentativa do passo 2 morreu em **1,7 segundo**, descartando as outras 989 bases.

O roteiro previa que o passo 2 terminasse em `FAIL` — mas pelo acúmulo de erros **por amostra**, após gravar o `report.json`, não por um aborto de setup antes de qualquer medição.

A certificação aceita imagens sem features e grava certificado só com pHash, e o caminho "smoke" do próprio teste já pulava essas bases com um `t.Logf`. O caminho do manifest agora faz o mesmo, e reporta quantas bases e amostras pulou. Os erros por amostra que o protocolo espera continuam acumulando e continuam fazendo o teste falhar depois de gravar o relatório.

11.2 O resumo conflava dois eventos opostos

O `tcc_summary.py` contava todo veredito "devolveu outro certificado" como erro de atribuição. Mas 993 dos 1.022 eram amostras `different_image`, que carregam os bytes do

par intactos e que o SHA-256 resolve corretamente para o certificado do próprio par — a resposta verdadeira.

Conflar os dois inflaria o número de erros de atribuição em **36 vezes**: 1.022 em vez dos 27 reais, e dos **zero** genuínos. Agora são linhas separadas, com detalhamento por base de origem logo abaixo — que foi o que apontou a `picsum_456` imediatamente.

12. Passo 4 — end-to-end contra a API

Este é o passo que valida se a simulação do passo 3 é fiel ao sistema real. O teste sobe os handlers HTTP reais num servidor de teste, persiste no PostgreSQL 16, e chama `POST /certificates` e `POST /certificates/verify` para cada amostra. Uma amostra conta como acerto apenas se o certificado devolvido for o da base correta.

item	valor
Duração	3 h 50 min 49 s (13.849 s)
Workers de verificação	4
Bases certificadas	993, 0 falhas
Amostras avaliadas	53.622
Erros HTTP ou de leitura	0
Veredito do teste	PASS (gates: precisão $\geq 0,95$ e recall $\geq 0,75$ em alta confiança)

A certificação usou o endpoint legado, por `allowUnattested = true`, sem atestação de hardware e sem ancoragem em blockchain. **O e2e mede a verificação, não o fluxo de captura atestado.** As quatro bases sem features ORB foram certificadas com sucesso, somente com pHash, confirmando em produção o comportamento descrito na seção 4.4.

12.1 Concordância com a simulação: exata

O critério de aceite do protocolo era que os totais ficassem a menos de **1 ponto percentual** da linha "Todas" da simulação. A divergência observada foi **zero**:

estrato	fonte	n	TP	FP	FN	TN
Todas	simulação em memória	53.622	24.549	1.887	10.206	16.980
Todas	e2e com PostgreSQL	53.622	24.549	1.887	10.206	16.980
Alta confiança	simulação em memória	16.881	8.783	3	154	7.941
Alta confiança	e2e com PostgreSQL	16.881	8.783	3	154	7.941
Limítrofe	simulação em memória	36.741	15.766	1.884	10.052	9.039
Limítrofe	e2e com PostgreSQL	36.741	15.766	1.884	10.052	9.039

A concordância não é apenas agregada. Comparando **família por família**, todas as 21 famílias produziram matrizes de confusão idênticas:

família	TP	FP	FN	TN	e2e = simulação
brightness	3.843	0	129	0	✓
color_invert	0	0	0	993	✓
content_overlay	0	0	0	3.972	✓
crop_border	798	0	2.181	993	✓
different_image	0	0	0	993	✓
downscale	1.319	0	3.646	0	✓
format_change	3.932	0	40	0	✓
grayscale	0	217	0	776	✓
heavy_crop	0	0	0	2.979	✓
hue_shift	0	546	0	3.426	✓
jpeg_recompress	5.785	0	173	0	✓
localized_recolor	0	284	0	709	✓
noise_light	1.951	0	35	0	✓
p3_as_srgb	834	0	159	0	✓
rotate_cardinal	1.734	0	1.245	0	✓
rotate_small	771	0	2.208	0	✓
saturation_boost	0	837	0	1.149	✓
sepia	0	3	0	990	✓
sharpen	981	0	12	0	✓
upscale	1.625	0	361	0	✓
whatsapp_like	976	0	17	0	✓

12.2 O que essa concordância estabelece

Três coisas, e vale separá-las.

Primeira: a diferença de ordem de empate não importa. A seção 6.3 registrou que o PostgreSQL deixa indefinida a ordem entre candidatos empatados em distância de Hamming, enquanto a simulação desempata pelo hash de conteúdo. A preocupação era que um empate resolvido de forma diferente levasse o pipeline a comparar candidatos em outra ordem e, eventualmente, a decidir diferente. Não aconteceu em nenhuma das 53.622 amostras. A explicação está na seção 7.6: com mediana de **1 candidato por consulta** e máximo de 12, empates que mudem o resultado são raríssimos — e quando o certificado correto está presente, ele está em primeiro lugar em 99,7% dos casos (30.454 de 30.549).

Segunda: a simulação é um modelo fiel. Como os resultados são idênticos, toda a análise fina do passo 3 — os diagnósticos de pré-filtro, o estágio decisor, a atribuição dos falsos negativos, a decomposição de latência — **transfere para o sistema real** sem ressalva. Isso é o que torna defensável usar o CSV da simulação como fonte dos números do capítulo, em vez de apenas as matrizes agregadas do e2e.

Terceira: as camadas de transporte e persistência não introduzem erro. O caminho e2e adiciona multipart HTTP, serialização JSON, a tabela `certificates`, a tabela `phash_bands` com 31.776 linhas, e a consulta `UNNEST` de 128 sondas de banda por verificação. Nenhuma dessas camadas alterou um único veredito. A reprodução da semântica do `FindCandidatesByPHashes` em memória estava correta.

12.3 Contrapartida: custo da pilha completa

medição	simulação (passo 3)	e2e (passo 4)	razão
Duração com 4 workers	2 h 34 min	3 h 51 min	1,50×
Amostras	54.615	53.622	—
Tempo por amostra	169 ms	258 ms	1,53×

O e2e é cerca de **50% mais lento por amostra**, e faz **menos** trabalho de análise — ele não calcula a comparação par a par nem os diagnósticos de pré-filtro, que a simulação calcula adicionalmente. O custo extra vem do transporte HTTP multipart, da serialização e das idas ao PostgreSQL. Como a seção 8 mostra que o pHash domina o custo computacional, essa sobrecarga de infraestrutura é a segunda maior componente da latência de produção.

13. Achados e recomendações consolidados

13.1 O que o sistema faz bem

1. **Precisão de alta confiança em 1,000**, com 3 eventos em 8.786 predições positivas, todos de uma única família (`sepia`) e todos com resíduo logo abaixo dos limiares. Para um certificador, esta é a propriedade que importa.
2. **Zero atribuições genuinamente erradas** em 52.629 oportunidades.
3. **Controle de imagem não certificada em 0 de 991**, limite superior 0,39%.
4. **O esquema de bandas do pré-filtro é eficiente**: só 0,2% das perdas vêm de ausência de colisão de banda; o resto é o limiar de Hamming, que é um parâmetro, não uma limitação estrutural.
5. **O pré-filtro LSH custa 0,5 ms**, validando a decisão de indexar bandas num índice composto.

6. **Rejeição robusta de edições de conteúdo:** `content_overlay` em todas as amplitudes, `heavy_crop`, `color_invert` e `different_image` fecham em 1,000.

13.2 Recomendações, em ordem de retorno

#	recomendação	efeito esperado	risco
1	Pré-computar a matriz de cossenos da DCT e explorar a separabilidade em <code>dct2D</code>	Latência de verificação de ~290 ms para 80–100 ms; hash idêntico	Nenhum — o valor do hash não muda
2	Reduzir <code>verifyTopK</code> de 64 para algo como 8 ou 16	Reduz o pior caso de latência; nenhuma perda de recall observada (máx. 12 candidatos em 54.615 consultas)	Precisa revalidação se o banco crescer ordens de magnitude
3	Proteger ou particionar <code>randSource</code> em <code>builders.go</code>	Torna a geração do dataset reprodutível em paralelo; elimina uma corrida de dados	Muda os bytes gerados de <code>noise_light</code> e <code>localized_recolor</code>
4	Detectar bases quase duplicadas no gerador, por distância pHash par a par	Evita colisões de verdade fundamental como a da seção 10	Nenhum
5	Registrar o hash do certificado devolvido no CSV do experimento	Torna autoexplicativa qualquer atribuição a "outro certificado"	Nenhum
6	Revisar os rótulos de amplitude pequena em <code>saturation_boost</code> e <code>hue_shift</code>	A inversão de monotonicidade (1,5× falha mais que 2,0×) sugere que o rótulo, não o sistema, está na fronteira errada	É decisão de taxonomia, exige rejustificação
7	Para melhorar rotação e corte, avaliar armazenar miniatura de referência	Permitiria refinamento sub-pixel do alinhamento	Decisão de arquitetura, com custo de armazenamento e implicações de privacidade

13.3 O que não deve ser feito

Relaxar `MaxCellDist` para fazer as rotações passarem. A seção 5.2 mostra que o gate está correto e o problema é a precisão do alinhamento. Elevar o limiar compraria recall de rotação ao custo de aceitar edições localizadas — exatamente o que o gate existe para pegar.

Aumentar `MaxPHashDistance` para recuperar os cortes de borda. A seção 7.6 mostra que `crop_border_15pct` (que deveria casar) e `crop_border_20pct` (que não deveria) têm taxas de recuperação quase idênticas no pré-filtro, 0,031 e 0,017. Um limiar mais folgado recuperaria os dois, trocando falsos negativos por falsos positivos na mesma proporção.

14. Limitações e ameaças à validade

1. **O estrato limítrofe foi definido a posteriori.** As taxas citadas nos comentários do registry foram observadas em 100 imagens do Picsum que estão entre as 993 deste benchmark. A seção 7.3 mostra que excluí-las não move a manchete, mas a dependência existe e deve ser declarada. Os IDs excluídos são uma **reconstrução por inferência**, não um registro daquela execução.
2. **Uma colisão de verdade fundamental no dataset**, documentada na seção 10.
3. **Fonte única de imagens.** Todas as bases vêm do Lorem Picsum, que é um recorte de fotografia profissional do Unsplash. Fotografias de celular, capturas de tela, documentos digitalizados e imagens sintéticas não estão representados, e são justamente os casos de uso de um certificador.
4. **Resolução única.** Todas as bases são 800×600. A interação entre o tamanho da célula do grid (que depende da resolução) e a precisão do alinhamento — causa raiz da seção 5.2 — não foi variada.
5. **O banco tem 993 certificados.** O comportamento do pré-filtro depende da densidade do banco, e a seção 7.6 observa no máximo 12 candidatos por consulta. Em escala de milhões, tanto a contagem de candidatos quanto a taxa de colisão crescem, e as conclusões sobre `verifyTopK` precisariam ser revalidadas.
6. **A simulação usa banco em memória.** A lógica do pré-filtro é reproduzida fielmente, mas a ordem entre candidatos empatados em distância difere do PostgreSQL. O passo 4 é o que estabelece se isso importa.
7. **O passo 4 certifica pelo endpoint legado**, com `allowUnattested = true`, sem atestação de hardware e sem ancoragem em blockchain. **O e2e mede a verificação, não o fluxo de captura atestado.**
8. **A latência foi medida numa máquina virtual** de 4 vCPU compartilhada (AMD EPYC-Rome sob KVM). Os números absolutos não transferem para hardware dedicado; as **proporções entre estágios** transferem.
9. **Acurácia não deve ser reportada isoladamente.** A taxonomia tem 34.755 positivos contra 18.867 negativos, inflando a métrica por construção.
10. **A correção do pHash altera a comparabilidade** com qualquer medição anterior a este benchmark nas famílias `format_change_bmp` e `format_change_tiff`.

15. Procedência dos arquivos

Todo número deste relatório sai de um destes arquivos, todos sob `results/`, e todos produzidos pelo código na tag `tcc-benchmark-v1` (exceto onde a seção 11 indica o contrário).

arquivo	origem	conteúdo
ambiente.txt	levantamento	commit, tag, toolchain, host, metadados do dataset
execucao.md	registro	procedência, desvios, avisos para o texto
manifest.json	passo 1	993 bases, 53.622 amostras, snapshot de limiares
source.lock.json	passo 1	catálogo travado do Picsum
calibration_base_ids.txt	reconstrução	as 100 bases que definiram o estrato limítrofe
duplicate_base_ids.txt	análise	o par de bases com conteúdo idêntico
tcc_results.csv	passo 3	54.615 linhas, 31 colunas, rodada de acurácia
resumo_tcc.md	passo 3	tabelas da rodada de acurácia
resumo_tcc_sem_duplicata.md	passo 3	o mesmo, sem o par duplicado
resumo_sem_calibracao.md	passo 3	o mesmo, sem as 100 bases de calibração
tcc_latency.csv	passo 3	2.732 linhas, rodada de latência em uma thread
resumo_latencia.md	passo 3	tabelas de latência
matrix_report.json	passo 2	matriz de confusão par a par
e2e_report.json	passo 4	matriz de confusão end-to-end
relatorio_completo.pdf	este documento	—
../logs/*.log	todas as etapas	saída bruta

15.1 Reprodução

```

git checkout tcc-benchmark-v1

# O host não precisa de Go nem OpenCV; o contêiner fornece ambos.
docker build -t aletheia-tcc -f <Dockerfile da seção 3.1> .

# Passo 1
go run -tags datasetgen ./cmd/datasetgen \
  --source picsum --count 1000 --seed 42 --workers 4 --out testdata/generated

# Passo 3 (medição principal)
TCC_OUT=results/tcc_results.csv TCC_WORKERS=4 \
  go test -tags integration -count=1 -timeout 0 -run TestTCC_PipelineSimulation ./tests/feature/
TCC_OUT=results/tcc_latency.csv TCC_WORKERS=1 TCC_STRIDE=20 \
  go test -tags integration -count=1 -timeout 0 -run TestTCC_PipelineSimulation ./tests/feature/

# Passo 2 (verificação cruzada)
go test -tags integration -count=1 -timeout 0 -run TestDataset_FullMatrix ./tests/feature/

# Passo 4 (end-to-end)
docker compose up -d postgres
E2E=1 DATASET_E2E_WORKERS=4 \
  go test -tags e2e -count=1 -timeout 0 -run TestE2E_GeneratedDataset_Matrix ./tests/e2e/

# Resumos
python3 scripts/tcc_summary.py results/tcc_results.csv --out results/resumo_tcc.md
python3 scripts/tcc_summary.py results/tcc_latency.csv --out results/resumo_latencia.md

```

A regeneração do manifest **não** regenera as variantes, que são reaproveitadas de disco. Por causa da corrida descrita na seção 3.3, uma geração totalmente nova produziria bytes diferentes nas famílias `noise_light` e `localized_recolor`.